

Improvements and Analysis of a K-Nearest Neighbours Approach for the Computation of the Region Occupied by a Set of Points

Sean Perman¹ and Mario Lopez¹

¹*Ritchie School of Engineering and Computer Science, University of Denver, Denver, Colorado, U.S.A.
sean.perman@du.edu*

Keywords: Concave Hull, Convex Hull, Polygon, Contour, K-Nearest Neighbours, Spatial Bucketing, Computational Geometry.

Abstract: Moreira and Santos introduced a k -nearest-neighbours variation of gift wrapping for constructing concave hulls. When construction fails, the algorithm discards its partial hull and restarts with a larger k . We show that one attempt with fixed k has a worst-case upper bound of $O(kn^2)$ and that repeated restarts give an $O(n^4)$ upper bound. We also prove termination: k is bounded by n , where the method becomes Jarvis march and returns the convex hull. The analysis identifies edge-intersection checking and repeated reconstruction as the principal costs. We explore uniform-grid spatial hashing to accelerate intersection checking while leaving the worst-case bound unchanged. We also explore checkpoint trimming, which uses convex-hull vertices as rollback points so construction can preserve part of the existing hull after a failure. Recovery of points left outside a closed hull remains future work. Finally, an eight-point counterexample demonstrates that increasing k does not always produce a less concave hull.

1 INTRODUCTION

Given a finite set of points scattered in the plane, what shape do they occupy? This question comes up whenever we have sampled locations—GPS pings, sensor readings, members of a cluster—and want a single boundary that describes the region they cover. The classic answer is the *convex hull*: the smallest convex polygon containing every point, computable in $O(n \log n)$ time by Graham scan (Graham, 1972) or by the gift-wrapping walk of Jarvis march (Jarvis, 1973). The convex hull is simple and unique, but its convexity is exactly the problem. Because it can never bend inward, it bridges over inlets, gaps, and the space between separated clusters, often enclosing large empty areas that are not really part of the data.

A *concave hull* relaxes this restriction. It is a polygonal boundary allowed to dip inward to follow the data more closely, capturing recesses and non-convex features the convex hull smooths over. There is no single “correct” concave hull, however: how tightly the boundary should hug the points is a matter of degree, so concave hull methods are controlled by a parameter that sets the trade-off between detail and smoothness.

This paper builds on one such method, the k -nearest neighbours algorithm of Moreira and Santos

(Moreira and Santos, 2007). Their idea is to run a gift-wrapping walk, but at each step to choose the next boundary vertex only from the k points nearest the current one, rather than from all points. The single parameter k controls everything: small k keeps the walk local and produces a detailed, concave boundary, while large k widens the choices and smooths the result back toward the convex hull. The method is appealing because it is intuitive, easy to implement, and steered by one understandable knob.

Its weakness is in how it recovers from trouble. The walk can reach a **dead end**, where every available next edge would cross the boundary built so far, or it can close up while leaving points stranded outside. In either case the original algorithm throws away the *entire* partial hull and starts over from scratch with $k + 1$. A single hard spot near the end of the walk can therefore discard almost all of the work already done, and this can repeat many times as k climbs.

This paper analyzes that cost and reduces it. We make four contributions. **First**, we give a runtime analysis the original paper left open, showing that one run with a fixed k costs $O(kn^2)$ in the worst case and that the restart mechanism can push the total as high as $O(n^4)$. **Second**, we replace the full restart with *checkpoint trimming*: using the convex-hull vertices as safe rollback points, the algorithm rewinds only to

the last checkpoint and regrows that section, preserving the good part of the boundary instead of discarding it. **Third**, we add *spatial bucketing*, a grid index that tests each new edge for intersection only against nearby edges, which speeds up the most expensive step in practice. **Fourth**, we evaluate these changes on real and synthetic datasets, where checkpoint trimming with bucketing runs up to $13\times$ faster than the restart baseline while discarding far less work. Along the way we also show, with a small eight-point counterexample, that k does not control concavity monotonically: increasing k can sometimes make the hull *more* concave, so k is best understood as a heuristic smoothness control rather than a strict dial.

The rest of the paper is organized as follows. Section 2 reviews convex and concave hulls and the Moreira–Santos algorithm in detail. Section 3 analyzes the runtime of the original method. The remaining sections develop checkpoint trimming and spatial bucketing, report our experiments, and discuss open questions and future work.

2 BACKGROUND AND THE MOREIRA–SANTOS ALGORITHM

2.1 The Concave Hull Problem

2.1.1 Convex Hulls and Their Limitations

Convex hull algorithms such as Graham scan (Graham, 1972) and Jarvis march (Jarvis, 1973) are classical tools for enclosing a planar point set with the smallest convex polygon containing all points. They are efficient, well understood, and produce a unique result for a given input set. However, that uniqueness comes at a cost: the resulting boundary often over-encloses the data and fails to capture important geometric structure such as indentations, narrow inlets, or separated lobes. For many applications, the convex hull is therefore too coarse to serve as a faithful representation of the region occupied by the points. In particular, when the data suggest a distinctly non-convex shape, a convex approximation may include large empty regions that are not meaningfully part of the underlying object or cluster.

2.1.2 Concave Hulls as Shape Descriptions

The concave hull problem asks for a polygonal boundary that more accurately reflects the perceived shape of a finite planar point set. Unlike the convex hull, a concave hull is intended to follow irreg-

ularities in the data, including local recesses and non-convex features. This makes it a more flexible shape descriptor in settings where geometric detail matters. A central difficulty, however, is that there is no single canonical concave hull. Different applications may require different tradeoffs between simplicity, smoothness, and fidelity to the point set. As a result, most concave hull methods are inherently parameterized, either explicitly or implicitly, and the chosen parameter controls how tightly the boundary conforms to the data (Galton and Duckham, 2006; Duckham et al., 2008).

2.1.3 Applications of Concave Hull Methods

Concave hull methods arise in a variety of settings where one seeks a meaningful outer boundary for spatial data. In geographic information systems, they can be used to estimate footprints or occupied regions from sampled locations. In clustering and data analysis, they help visualize or extract non-convex cluster boundaries. They are also relevant in spatial reasoning, where the goal is not merely to enclose points, but to describe the region those points collectively occupy (Galton and Duckham, 2006).

2.2 The Moreira–Santos K-Nearest Neighbours Algorithm

2.2.1 Main Idea

Moreira and Santos (Moreira and Santos, 2007) proposed a simple and intuitive concave hull algorithm based on the idea of modifying Jarvis march (Jarvis, 1973). Instead of selecting the next hull vertex from all points, their method restricts attention to the k -nearest neighbours of the current vertex. Among these **candidates**, it chooses the one that creates the largest right-hand turn without introducing an intersection with the existing hull. This restriction makes it possible to construct non-convex boundaries while preserving the local, step-by-step flavor of gift wrapping (Jarvis, 1973). The parameter k governs how local the search is: smaller values tend to produce tighter and more detailed hulls, while larger values yield smoother and more convex boundaries.

The algorithm begins by selecting an extreme point (minimum y -coordinate) as the starting vertex. From the current hull vertex, it computes the k -nearest neighbours that have not been excluded from consideration. These candidates are then ordered according to the turning angle relative to the previous hull edge. A greedy approach is then used to select a **valid candidate** that produces the largest right-hand turn without intersecting the existing hull. This process repeats

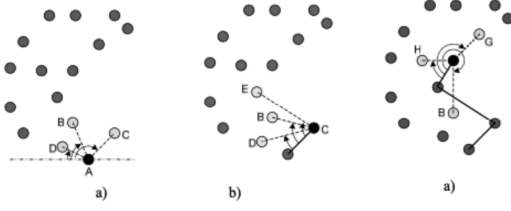


Figure 1: The k -nearest neighbours approach.

until the walk returns to the starting point, thereby closing the polygon.

2.2.2 Failure Cases and Restart Logic

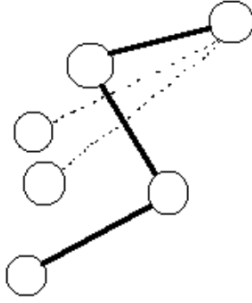


Figure 2: All candidate edges intersect the current hull.

The algorithm has two main failure cases. First, there may be no valid candidates among the current k -nearest neighbours: each candidate edge intersects the existing partial hull, so the algorithm cannot safely extend the boundary (**dead end**). Second, after the hull has been closed, some input points may still remain outside the polygon. In that case, the computed hull fails to enclose the full point set (**excluded points**).

2.2.3 The Role of the Parameter k

The parameter k is the central control variable in the algorithm. When k is small, candidate selection is highly local, allowing the boundary to follow fine geometric detail and concave features. When k is larger, the set of candidate vertices broadens, and the resulting hull tends to be smoother and closer to a convex enclosure. In the case where $k = n$, large enough that all remaining points are considered at each step, the method is identical to the Jarvis-march search and produces the convex hull. An important feature of the method is that k is dimensionless: it is not a geometric distance threshold, but rather a count of neighbours. This distinguishes it from approaches such as alpha

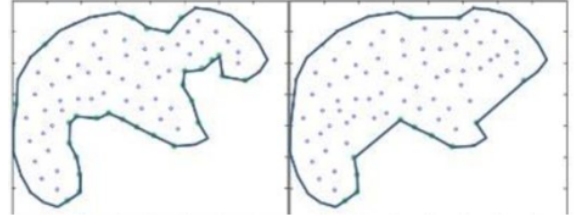


Figure 3: (left = low k), (right = high k).

shapes, where the main parameter is directly tied to spatial scale and may be sensitive to sampling density (Edelsbrunner et al., 1983). In the Moreira–Santos method, the parameter has a more combinatorial interpretation, which can make it easier to reason about across differently scaled datasets.

2.2.4 Comparison with Other Concave Hull Approaches

Compared with alpha-shape-based methods and related shape reconstruction techniques, the Moreira–Santos algorithm has the advantage of conceptual simplicity and a parameter that is not directly tied to metric scale. In contrast, methods based on geometric thresholds often require careful tuning relative to point spacing or sampling density (Edelsbrunner et al., 1983; Liao et al., 2021). The k -nearest-neighbours approach is therefore appealing both practically and theoretically. It offers a direct geometric construction, adapts automatically by increasing k when necessary, and provides a natural bridge between highly detailed concave boundaries and more convex approximations. These properties make it a useful baseline for the improvements developed in the later sections of this paper.

3 RUNTIME ANALYSIS OF THE ORIGINAL ALGORITHM

3.1 Cost of One Successful or Failed Run with Fixed k

We first analyze one execution of the original Moreira–Santos algorithm with a fixed neighbour parameter k . Let n denote the number of input points, and let s denote the number of steps completed before that run either closes a hull or fails and triggers a restart. Since each non-closing step adds a new hull vertex and removes a point from further consideration, we have $s = O(n)$. At each step, the algorithm performs three main operations:

Nearest-neighbour search. At each step, the algorithm computes the k -nearest neighbours of the current point. The original uses a pre-made nearest-neighbour function in Mathematica, which internally uses spatial indexing. Even so, for worst-case analysis we will use the upper bound:

$$T_{\text{knn}}(i) = O(n).$$

Sorting the candidates. Once the k nearest neighbours have been selected, they are sorted by angle relative to the previous hull direction. Sorting k items costs

$$T_{\text{sort}}(i) = O(k \log k).$$

Intersection checking. Suppose the partial hull currently contains i vertices. A candidate edge must be checked against $O(i)$ existing hull edges. Since the algorithm may test as many as k candidates before finding a valid one or concluding that no valid candidate exists, the intersection-checking cost at step i is

$$T_{\text{check}}(i) = O(ki).$$

Therefore, the total cost at step i is

$$T(i) = O(n) + O(k \log k) + O(ki).$$

Summing over all s steps in the run gives

$$C_{\text{run}}(k) = \sum_{i=1}^s \left(O(n) + O(k \log k) + O(ki) \right).$$

Separating the three contributions:

$$C_{\text{run}}(k) = O(sn) + O(sk \log k) + O\left(k \sum_{i=1}^s i\right).$$

Using the triangular-sum identity, $\sum_{i=1}^s i = O(s^2)$, we obtain

$$C_{\text{run}}(k) = O(sn) + O(sk \log k) + O(ks^2).$$

Since $s = O(n)$, this yields

$$C_{\text{run}}(k) = O(n^2) + O(nk \log k) + O(kn^2).$$

A completed run also performs a final containment check to verify that all points lie inside the constructed polygon. In the worst case, this adds at most $O(n^2)$, which does not change the asymptotic bound above. Since $k \geq 1$, the term $O(kn^2)$ dominates, giving a single run with fixed k worst-case cost:

$$C_{\text{run}}(k) = O(kn^2).$$

3.2 Cost of Restarts

The original algorithm may restart with a larger value of k in two cases: first, when all current candidates would produce an intersecting edge (**dead end**), and second, when a closed hull is found but at least one input point lies outside the resulting polygon (**excluded points**). In either case, the algorithm discards the partially constructed hull and restarts from the beginning with $k + 1$. Suppose the algorithm begins with k_0 and eventually succeeds only after trying the sequence $k_0, k_0 + 1, k_0 + 2, \dots, K$. Then the total cost is:

$$C_{\text{total}} = \sum_{k=k_0}^K C_{\text{run}}(k) = O\left(n^2 \sum_{k=k_0}^K k\right).$$

In the worst case, the algorithm restarts may increase k until $K = n - 1$. Therefore,

$$C_{\text{total}} = O\left(n^2 \sum_{k=1}^{n-1} k\right) = O(n^2 \cdot n^2) = \boxed{O(n^4)}.$$

3.3 Main Takeaways from the Runtime Analysis

The runtime analysis highlights two main sources of cost in the original Moreira–Santos algorithm. First, within a single run, the dominant expense comes from edge-intersection checking. As the partial hull grows, each new candidate edge may need to be tested against more previously constructed hull edges, producing the triangular-sum behavior that drives the per-run cost. The second, and more serious, issue is the restart mechanism. When the algorithm encounters a difficult local configuration, it discards the entire partial hull and starts again with a larger value of k . This means that even if most of the hull was constructed successfully, a tricky section near the end can force the algorithm to throw away almost all previous work. As a result, the ratio of wasted work to retained work can become very large, and the total runtime can grow rapidly. In this sense, the main weakness of the original algorithm is not only that each run is expensive, but that local failures trigger repeated global reconstruction.

4 MONOTONICITY OF K

The parameter k is usually described as a smoothness control: the intuition from Section 2 is that a larger k broadens the candidate set at each step, and so should yield a smoother boundary that sits closer to the convex hull. It is natural to ask whether this relationship is *monotone*—that is, whether increasing k can

only ever make the hull less concave, never more. In this section we show that it cannot. After defining a simple numerical measure of concavity, we exhibit a small point set on which raising k by one produces a strictly *more* concave hull, even though both hulls enclose every input point. The lesson is that k is best understood as a heuristic control over smoothness rather than a parameter with a guaranteed monotone effect.

4.1 A Measure of Concavity

To compare hulls for different values of k , we use the area ratio

$$\rho(H) = \frac{\text{area}(H)}{\text{area}(\text{conv}(S))},$$

where H is the hull produced for the point set S and $\text{conv}(S)$ is the convex hull of S . Since every valid hull must contain all points of S ,

$$0 < \rho(H) \leq 1,$$

with $\rho(H) = 1$ exactly when H is the convex hull. Smaller values of $\rho(H)$ mean that the boundary cuts away more empty space from the convex hull, so the hull is more concave. If k controlled smoothness monotonically, then increasing k should never make $\rho(H)$ smaller. The following example shows that this is not always true.

4.2 A Counterexample

Consider the eight-point set S in Table 1. Its convex hull is the triangle with vertices

$$(0.12534, 0.79581), \quad (0.83024, 0.17598), \\ (0.99603, 0.94980),$$

and $\text{area}(\text{conv}(S)) = 0.32411$.

Table 1: The eight-point counterexample set S .

x	y
0.99603	0.94980
0.12534	0.79581
0.83024	0.17598
0.57695	0.84059
0.93863	0.70464
0.63777	0.76837
0.83103	0.86563
0.50715	0.63669

Running the algorithm with $k = 5$ produces the hull in Figure 4:

$$\text{area}(H_5) = 0.32223, \\ \rho(H_5) = \frac{0.32223}{0.32411} \approx 0.9942.$$

Running the same algorithm with $k = 6$ produces the hull in Figure 5:

$$\text{area}(H_6) = 0.26187, \\ \rho(H_6) = \frac{0.26187}{0.32411} \approx 0.8079.$$

The ratio drops from 0.9942 to 0.8079 when k increases by one, so the larger value of k produces a strictly more concave hull. Both hulls enclose all eight input points, ruling out the degenerate case where a larger k simply causes the walk to close early and miss points.

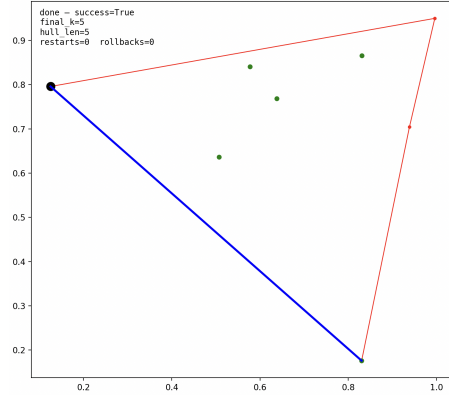


Figure 4: Hull produced with $k = 5$: $\rho \approx 0.9942$.

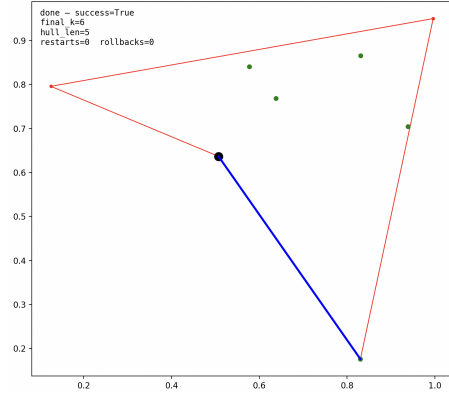


Figure 5: Hull produced with $k = 6$: $\rho \approx 0.8079$.

The difference comes from how the unused points affect the later k -nearest neighbour sets. In the $k = 5$ run, the algorithm cannot jump directly from the lowest point of the triangle to the top-right corner, so it first uses an extra outer point on the right side. Since that point has already been used, it is removed from the candidate set later, allowing the final closing point to appear among the nearest candidates. In the $k = 6$ run, the algorithm skips that extra outer point at first. Later, the skipped point is still unused, so it remains in the k -nearest neighbour set and effectively takes the

place of the final closing point. The walk therefore does not see the closing point at the moment when the $k = 5$ run does, and it is forced to continue along a more inward path. Thus the larger value of k changes which points remain available later, producing a more concave hull for $k = 6$ than for $k = 5$.

5 CHECKPOINT-BASED TRIMMING MODIFICATION

5.1 Motivation

The checkpoint mechanism is motivated by the observation that the restart rule in the original algorithm is often too aggressive. When the algorithm reaches a **dead end** (all candidate next edges intersect the current hull), the algorithm abandons the entire partial solution and begins again with a larger value of k . But this suggests two important questions. **First**, can the amount of discarded work be reduced? **Second**, how can the algorithm determine which part of the current hull is responsible for the dead end? If the failure is local, then only a suffix of the hull may need to be reconsidered, while the earlier prefix can be preserved.

5.2 Convex Hull Vertices as Checkpoints

We propose a checkpoint-based version of the algorithm; we choose the checkpoints to be the vertices of the convex hull of the input set, listed in cyclic order. This choice is useful for two main reasons.

First, convex hull vertices are extreme points of the data set, so they are natural outer anchors for any hull intended to enclose all input points. Under our hull model, the boundary is constructed from points of the input set and is required to enclose the full data set. Since the convex hull is the smallest convex set containing the input, its vertices represent the outermost points of the set. For this reason, they are the safest points to use as rollback targets: trimming back to a convex hull vertex preserves a prefix of the walk that is still anchored on the true outer structure of the point set.

Second, convex hull vertices come with a canonical cyclic order around the data set. This makes them especially useful for checkpoint-based recovery, since they partition the boundary into well-defined regions between consecutive checkpoints. The walk is therefore expected to encounter these checkpoints in order. If the algorithm reaches a later checkpoint before reaching the expected next one, then it has skipped

part of the outer boundary structure. In that case, the walk has either left some extreme point outside the current hull, or has moved into a configuration that cannot be completed correctly without backtracking. This makes checkpoint ordering a natural consistency condition. Together, these properties make convex hull vertices good checkpoints. They are guaranteed outer boundary anchors, they impose a meaningful global order on the walk, and they provide safe locations to which the algorithm can trim when a local dead end occurs.

5.3 The Trim Operation

Before describing the trim operation, we define a few terms that will be used throughout this section.

Definitions

A **checkpoint** is a selected convex hull vertex used as a safe rollback location. The checkpoints are ordered cyclically around the point set. A **region** is the portion of the boundary between two consecutive checkpoints. Each region can have its own local value of k . A **dead end** occurs when the current hull endpoint has no valid continuation because every candidate next edge intersects the existing partial hull. A **rollback checkpoint** is the checkpoint to which the algorithm trims the hull after a dead end is detected.

Step 1: Dead End (Trigger)

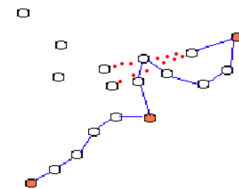


Figure 6: A dead end: every candidate next edge from the current endpoint intersects an existing hull edge.

The trim operation is triggered when the current hull endpoint reaches a **dead end**, meaning that every candidate next edge intersects an existing hull edge. At this point, the walk cannot be safely extended using the current local value of k .

Step 2: Finding the Rollback Checkpoint

To determine how far back the hull should be trimmed, the algorithm considers the highest-priority failed candidate under the usual angle ordering. It then finds the earliest hull edge intersected by that

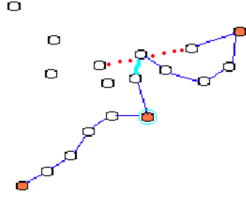


Figure 7: Tracing back from the first conflicting hull edge to the most recent checkpoint, which becomes the rollback target.

candidate. Starting from that conflict location, the algorithm walks backward along the current hull until it reaches the most recent checkpoint. This checkpoint becomes the rollback target.

Step 3: Trim and Increase k

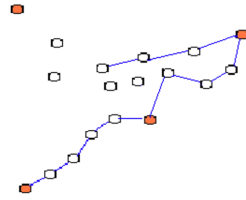


Figure 8: Trimming the suffix back to the rollback checkpoint and resuming the walk with an increased value of k .

Once the rollback checkpoint has been identified, the algorithm removes the suffix of the hull after that checkpoint and returns the removed points to the active data set so that they may be reconsidered. It then increases k only for the region beginning at that checkpoint, rather than increasing k globally for the entire hull. The walk then resumes from the checkpoint using the preserved prefix of the hull.

Out-of-order checkpoints

A trim is also triggered when the walk reaches a checkpoint out of order. the boundary should encounter checkpoints in order as it moves around the point set. If the walk reaches a later checkpoint before reaching the expected next checkpoint, then it has skipped part of the outer boundary structure. In this case, the newly constructed suffix is rejected and the algorithm trims back to the current checkpoint. The points removed by this trim are returned to the active data set, and the local value of k for the affected region is increased before the walk resumes. This prevents the algorithm from wrapping around the point set incorrectly and helps preserve the intended region-by-region traversal of the hull.

Takeaways

This modification prevents the algorithm from restarting from the beginning every time a local failure occurs. Instead, it preserves the valid prefix of the hull and recomputes only the section that is likely responsible for the problem. Trimming may be triggered either by a dead end, when no candidate edge can be added without intersection, or by reaching a checkpoint out of order, which indicates that the walk has skipped part of the intended outer boundary structure. In both cases, the algorithm responds locally by preserving useful work.

Global versus per-region k

There are two natural ways to update k after a trim. The first is to give each region its own local value of k . In this version, only the region that fails has its k increased. This keeps the algorithm more adaptive: simple regions can keep a small k , while difficult regions can use a larger one. This choice affects the shape of the final hull. A smaller k usually makes the walk more local, which can preserve more concave detail. A larger k gives the algorithm more candidate points to choose from, but it also tends to smooth the boundary and move the result closer to the convex hull. Thus, per-region k can produce a more locally detailed boundary, because not every part of the hull is forced to use the same large value of k . The second option is to use one global value of k for the whole algorithm. When any region fails, the shared value of k is increased and used everywhere. This is less locally adaptive, because all regions use the same value whether they need it or not. However, it can be faster because the algorithm does not have to increase k separately in each region. Therefore, the choice is a tradeoff. Per-region k gives more local control and may preserve more concavity, while global k is simpler and often faster, but can produce a smoother and less locally detailed hull.

6 SPATIAL BUCKETING IMPROVEMENT

The runtime analysis shows that edge-intersection checking is one of the main costs of the algorithm. When the hull has many edges, a naive implementation checks each new candidate edge against every edge already in the partial hull. Most of these checks are unnecessary, because a candidate edge can only intersect hull edges that are geometrically nearby. To reduce this cost, we use a simple spatial bucketing

method. The plane is divided into a uniform grid of square cells. As the hull is constructed, each hull edge is stored in every grid cell that it passes through. Then, when the algorithm tests a new candidate edge, it first finds the grid cells crossed by that candidate edge and only checks against hull edges stored in those same cells. This does not change the intersection test itself. The algorithm still performs exact segment-intersection checks; the grid only reduces the number of edges that need to be tested. Since every hull edge is registered in each cell it crosses, an intersecting edge will still be found. In this way, bucketing avoids many unnecessary comparisons without changing the correctness of the intersection check.

6.1 Effect on Running Time

Let h be the number of hull edges built so far. In the naive version, checking one candidate edge may require comparing it against all h existing hull edges. With spatial bucketing, the candidate edge is compared only against the edges in the grid cells it crosses. If there are i such nearby edges, then one candidate costs $O(i)$ intersection checks instead of $O(h)$. In the worst case, this does not improve the asymptotic bound. For example, many edges could fall into the same grid cell, making i almost as large as h . However, for typical point sets, hull edges are spread across the plane, so a candidate edge usually interacts with only a small local part of the hull. Thus, spatial bucketing should be understood as a practical constant-factor improvement rather than a new worst-case complexity result. Spatial bucketing was useful in all of our tests and reduced the running time for every variant we measured. By limiting intersection checks to nearby hull edges, it avoids many unnecessary comparisons during hull construction. This makes each candidate-edge test cheaper while still using the same exact intersection test.

7 TERMINATION

It is not obvious that checkpoint trimming halts: each trim discards part of the hull and resumes, so in principle the algorithm could trim and regrow forever. It cannot, and the reason is the parameter k itself.

Consider the total of the per-region parameters, $\Phi = \sum_R k_R$. Every trim—whether caused by a dead end or by reaching a checkpoint out of order—raises k_R for at least one region and never lowers any of them, so each trim strictly increases Φ . The regions are fixed in advance (one per convex-hull vertex), and each k_R can never exceed n , so Φ has a finite ceiling

and can grow only finitely often. There can therefore be only finitely many trims. Between two consecutive trims the walk merely adds vertices, each of which consumes a point, so at most $O(n)$ steps pass before it trims again or closes. Finitely many trims, each followed by a bounded amount of work, means the algorithm halts.

The cap $k_R \leq n$ is what makes this ceiling real. When a region reaches $k_R = n$, every remaining point is a candidate at each step, so the walk there is exactly the convex-hull gift-wrapping march of Jarvis, which never dead-ends and never skips a checkpoint—so that region can never trigger another trim. In the worst case every region rises to $k_R = n$ and the method degenerates into a convex hull algorithm, which always succeeds. (For the global- k variant the same argument applies to the single shared value of k .)

8 EXPERIMENTAL EVALUATION

We recreated the original algorithm in a single configurable program, with options to turn different features on and off for testing. This made it possible to run each variant on the same input data while changing only one feature at a time. The baseline is the original restart method: when the algorithm fails, it restarts with a larger value of k . This baseline also keeps the original all-points-inside check, so its final hull always contains every input point. We used four real geographic datasets: world capitals ($n = 194$), contiguous U.S. cities ($n = 219$), gray-wolf sightings ($n = 1054$), and coast-redwood sightings ($n = 1190$). We also used two synthetic datasets: points sampled uniformly from a disk, as in the original paper (Moreira and Santos, 2007), and an eight-pointed star designed to create difficult concavities and force dead ends. The synthetic datasets were tested up to $n = 50,000$ over five random seeds. For each run, we measured wall-clock time, the number of failures, the total number of hull edges discarded during failures, and the number of points left outside the final hull. We call the discarded edges *wasted work*, since those edges were computed but later thrown away.

8.1 Spatial Bucketing

Spatial bucketing improved the cost of intersection testing. On every dataset, the bucketed version produced the same hull as the naive intersection test, but ran faster. The speedup was larger on harder inputs, where many candidate edges had to be tested. Bucketing was especially helpful for checkpoint trimming. Trimming avoids full restarts, but each rollback still

Table 2: Restart baseline versus checkpoint trimming with per-region (local) and global k , all using convex-hull checkpoints and the bucketed intersection test. Time is in milliseconds; “wasted” is the total hull edges discarded across all failures; “out” is the number of points left outside the final hull. Each “speedup” is the baseline’s running time divided by that variant’s (higher is faster); values below $1\times$ indicate the variant is *slower* than restart. Synthetic rows are at $n = 50,000$.

Dataset	n	Restart (original)			Trim + local k				Trim + global k			
		time	wasted	out	time	wasted	out	speedup	time	wasted	out	speedup
capitals	194	64	1134	0	33	641	0	$1.9\times$	22	347	0	$2.9\times$
cities	219	4.7	72	0	3.6	16	0	$1.3\times$	3.5	16	0	$1.3\times$
wolf	1054	152	1890	0	235	3360	9	$0.6\times$	80	806	5	$1.9\times$
redwood	1190	3262	15007	0	1605	14278	12	$2.0\times$	255	3304	0	$12.8\times$
unit disk	50k	970	672	0	1444	1194	16	$0.7\times$	793	98	1	$1.2\times$
star	50k	2783	4369	0	2886	5730	8	$0.96\times$	1301	1619	2	$2.1\times$

has to rebuild and re-check part of the hull. As a result, trimming performs many intersection tests, so making each test cheaper has a large effect. For this reason, all comparisons below use the bucketed intersection test. This keeps the intersection method fixed and lets us focus on the effect of the recovery strategy.

8.2 Checkpoint Trimming

The main question was whether checkpoint trimming saves work compared with a full restart. The result depended on how k was updated.

We first tried a per-region strategy, where each region between convex-hull checkpoints maintained its own value of k . This was intended to keep simple regions detailed while increasing k only in difficult regions. In practice, however, it created too many small rollbacks. Since the convex hull can contain many checkpoints, many regions had to rediscover the same larger value of k independently. On the redwood dataset, this produced 372 rollbacks. Even though each rollback was local, the total cost was worse than restarting.

The faster approach was to use a single global value of k . When any region failed, the global k was increased and then reused everywhere. This avoided repeated local increases and made trimming consistently faster. Table 2 compares this global- k trimming version with the restart baseline; both use spatial bucketing.

This also suggests a trade-off between speed and boundary detail. A per-region k may preserve more local concavity, because easy regions can remain at smaller values of k . A global k is faster, but once k increases, that larger value is applied to the entire hull, which may smooth regions that did not need it. Our experiments measure running time, wasted work, and excluded points, so we only claim the speed result here. A direct comparison of boundary fidelity is left for future work.

In practice, the best way to use checkpoint trimming may depend on the desired balance between

speed and boundary detail. If the main goal is speed, it may be better to start with a moderately larger initial value of k , rather than beginning at the smallest possible value and forcing the algorithm to discover a usable k through many failures. Another option is to replace linear growth, $k \leftarrow k + 1$, with a faster schedule such as geometric growth. For example, after a failed region, the algorithm could update k by $k \leftarrow \lceil rk \rceil$ for some constant $r > 1$. This would reduce the number of failed attempts needed to reach a workable value of k . The trade-off is that larger jumps in k may overshoot the smallest valid value, producing a smoother and less detailed hull. Thus, applications that prioritize runtime may prefer a larger initial k or faster growth schedule, while applications that prioritize boundary detail may prefer slower growth.

8.3 Excluded Points

The checkpoint trimming feature was tested without the second failure case: excluded points, meaning points left outside the hull after the hull has closed. This was intentional. The goal of these experiments was to isolate checkpoint trimming and measure how much work it saves when the algorithm reaches local dead ends. In other words, this gives a baseline for how the feature performs before adding recovery logic for excluded points. We record how many excluded points there are after each run terminates in Table 2.

9 CONCLUSION

We revisited the Moreira–Santos k -nearest neighbours concave hull algorithm with two goals: to understand why it can be slow, and to make it faster without changing the shapes it produces. On the analysis side, we supplied the worst-case runtime the original paper left open, showing that a single run with fixed k costs $O(kn^2)$ and that the restart-from-scratch

recovery rule can compound this to $O(n^4)$. The cause is structural: a single dead end late in the walk forces the algorithm to throw away an almost-complete hull and begin again.

Our two improvements target the two costs the analysis identified. Checkpoint trimming replaces the global restart with a local rollback to the nearest convex-hull vertex, so the algorithm keeps the portion of the boundary it has already built correctly and rebuilds only the troublesome section. Spatial bucketing shrinks the dominant edge-intersection test by comparing each new candidate edge only against nearby hull edges. Used together—with convex-hull checkpoints and a single global k shared across regions—these changes ran up to $13\times$ faster than the restart baseline on our real and synthetic datasets while discarding far less work, and the bucketed intersection test produced exactly the same hulls as the naive one.

These gains come with honest limits. Our checkpoint version drops the original guarantee that every input point ends up inside the hull, so it can still leave a small number of points outside; recovering that guarantee without surrendering the speedup is the most important loose end. We also showed, with an eight-point counterexample, that k does not trade off smoothly against concavity—increasing k can make a hull *more* concave—so k is best treated as a heuristic smoothness control rather than an exact dial. Closing the excluded-points gap, proving that the modified algorithm always terminates, and measuring boundary quality rather than only speed are the natural next steps, which we outline below.

10 FUTURE WORK

Several directions remain for improving and evaluating the modified Moreira–Santos concave hull algorithm.

Excluded-points recovery. The most important next step is to handle the second failure case: points left outside a closed hull. The original algorithm handles this case by restarting with a larger value of k .

Amortized analysis of the full algorithm. A useful future direction is an amortized runtime analysis of the full algorithm. Rather than only bounding the cost of a single step, this analysis would measure the total work spent building, discarding, and rebuilding hull edges as k increases. This would make the comparison between full restarts and checkpoint trimming more precise. Full restarts may repeatedly

rebuild large parts of the same hull, while checkpoint trimming is intended to preserve most completed work and only redo the affected portion. Proving whether this reduces the amortized asymptotic cost, rather than only improving runtime in practice, remains open.

Parallel construction. Since convex-hull checkpoints divide the boundary into regions, a natural future direction is to grow different regions of the hull in parallel and then stitch them together at shared checkpoints. The main challenges are maintaining a consistent value of k across regions and handling intersection tests near region boundaries. A parallel version may require either a per-region k schedule or synchronization between neighboring regions.

Richer candidate selection. The current algorithm chooses the next hull vertex greedily, using only the turning angle among the k -nearest candidates. Future work could add additional heuristics to make this choice less local. For example, candidate points could be scored using a combination of turn angle, distance, progress toward the next checkpoint, or whether a region has failed before. Heuristics could also be used earlier, when selecting which points should be considered as candidates in the first place. These changes may reduce dead ends and unnecessary trimming, but they would introduce new parameters and would need to be tested for both runtime and hull quality.

Evaluation breadth. Finally, the algorithm should be tested on a wider range of inputs. In particular, non-uniformly dense point sets and separated clusters are important because they are a documented hard case for the original algorithm. Degenerate inputs, such as collinear points, lattice-like point sets, and cocircular points, would also help test robustness. Adding a boundary-quality metric would make it possible to compare not only running time, but also how well each version preserves the intended concave shape of the data.

REFERENCES

- Duckham, M., Kulik, L., Worboys, M., and Galton, A. (2008). Efficient generation of simple polygons for characterizing the shape of a set of points in the plane. *Pattern Recognition*, 41(10):3224–3236.
- Edelsbrunner, H., Kirkpatrick, D., and Seidel, R. (1983). On the shape of a set of points in the plane. *IEEE Transactions on Information Theory*, 29(4):551–559.
- Galton, A. and Duckham, M. (2006). What is the region occupied by a set of points? In *GIScience*, pages 81–98.

- Graham, R. L. (1972). An efficient algorithm for determining the convex hull of a finite planar set. *Information Processing Letters*, 1(4):132–133.
- Jarvis, R. A. (1973). On the identification of the convex hull of a finite set of points in the plane. *Information Processing Letters*, 2(1):18–21.
- Liao, Z., Liu, J., Shi, G., and Meng, J. (2021). Grid partition variable step alpha shapes algorithm. *Mathematical Problems in Engineering*.
- Moreira, A. and Santos, M. Y. (2007). Concave hull: A k-nearest neighbours approach for the computation of the region occupied by a set of points. In *Proceedings of the International Conference on Computer Graphics Theory and Applications (GRAPP)*, pages 61–68.